Practical 9

Aim: A system has RAM which can store four pages simultaneously to satisfy page requests. Write a Program to calculate percentage page hit and page fault if the system uses Optimal page replacement technique for the page references entered by the user i.e. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (No of Frames=3).

Code:

```c
#include <stdio.h>

int findOptimal(int pages[], int frames[], int n, int index, int frameCount) {
    int farthest = index, pos = -1;
    for (int i = 0; i < frameCount; i++) {
        int j;
        for (j = index; j < n; j++) {
            if (frames[i] == pages[j]) {
                if (j > farthest) {
                    farthest = j;
                    pos = i;
                }
                break;
            }
        }
        if (j == n) {
            return i; // Page not used again, replace this
        }
    }
    return (pos == -1) ? 0 : pos;
}

int main() {
    int pages[] = {1,2,3,4,1,2,5,1,2,3,4,5};
    int n = sizeof(pages)/sizeof(pages[0]);
    int frameCount = 3;
    int frames[frameCount];
    int count = 0, hits = 0, faults = 0;

    for (int i = 0; i < frameCount; i++) frames[i] = -1;

    for (int i = 0; i < n; i++) {
        int flag = 0;
        for (int j = 0; j < frameCount; j++) {
            if (frames[j] == pages[i]) {
                hits++;
                flag = 1;
                break;
            }
        }
        if (!flag) {
            if (count < frameCount) {
                frames[count++] = pages[i];
            } else {
                int pos = findOptimal(pages, frames, n, i+1, frameCount);
                frames[pos] = pages[i];
            }
            faults++;
        }
    }

    printf("Total Requests: %d\n", n);
    printf("Page Hits: %d\n", hits);
```

```c
int main() {
    int pages[] = {1,2,3,4,1,2,5,1,2,3,4,5};
    int n = sizeof(pages)/sizeof(pages[0]);
    int frameCount = 3;
    int frames[frameCount];
    int count = 0, hits = 0, faults = 0;

    for (int i = 0; i < frameCount; i++) frames[i] = -1;

    for (int i = 0; i < n; i++) {
        int flag = 0;
        for (int j = 0; j < frameCount; j++) {
            if (frames[j] == pages[i]) {
                hits++;
                flag = 1;
                break;
            }
        }
        if (!flag) {
            if (count < frameCount) {
                frames[count++] = pages[i];
            } else {
                int pos = findOptimal(pages, frames, n, i+1, frameCount);
                frames[pos] = pages[i];
            }
            faults++;
        }
    }

    printf("Total Requests: %d\n", n);
    printf("Page Hits: %d\n", hits);
    printf("Page Faults: %d\n", faults);
    printf("Hit Percentage: %.2f%%\n", (hits * 100.0) / n);
    printf("Fault Percentage: %.2f%%\n", (faults * 100.0) / n);

    return 0;
}
```

Output:

```
HP@LAPTOP-9M3IBDT5 MSYS ~
$ cd

HP@LAPTOP-9M3IBDT5 MSYS ~
$ nano optimal.c

HP@LAPTOP-9M3IBDT5 MSYS ~
$ gcc optimal.c

HP@LAPTOP-9M3IBDT5 MSYS ~
$   ./optimal
-bash: ./optimal: No such file or directory

HP@LAPTOP-9M3IBDT5 MSYS ~
$ gcc optimal.c -o optimal

HP@LAPTOP-9M3IBDT5 MSYS ~
$ ./optimal
Total Requests: 12
Page Hits: 5
Page Faults: 7
Hit Percentage: 41.67%
Fault Percentage: 58.33%

HP@LAPTOP-9M3IBDT5 MSYS ~
$
```